

Capítulo 1

Aspectos Preliminares



Konrad Zuse

Konrad Zuse projetou uma série de computadores eletromecânicos entre 1936 e 1944 na Alemanha. Em 1945, projetou uma linguagem de programação algorítmica completa, a Plankalkül, jamais implementada, e sua descrição completa nem mesmo foi publicada até 1972.

- 1.1 Motivos para Estudar os Conceitos de Linguagens de Programação
- 1.2 Domínios de Programação
- 1.3 Critérios de Avaliação da Linguagem
- 1.4 Influências sobre o Projeto da Linguagem
- 1.5 Categorias de Linguagem
- 1.6 *Trade-Offs* no Projeto da Linguagem
- 1.7 Métodos de Implementação
- 1.8 Ambientes de Programação

Antes de iniciarmos nossa exposição dos conceitos de linguagens de programação, devemos considerar alguns aspectos. Primeiro, discutiremos alguns motivos pelos quais os estudantes de ciências da computação e os desenvolvedores de software profissionais devem estudar os conceitos gerais de projeto e de avaliação das linguagens. Essa discussão é valiosa para os que acreditam ser o conhecimento funcional de uma ou de duas linguagens de programação suficiente para os cientistas da computação. Os principais domínios de programação serão descritos brevemente. Em seguida, uma vez que o livro avalia recursos de linguagem, apresentaremos uma lista de critérios por meio dos quais se pode fazer julgamentos. As duas principais influências sobre o projeto da linguagem, sobre a arquitetura de máquina e sobre as metodologias de projeto de programa serão, então, discutidas. Depois, descreveremos alguns dos principais *trade-offs* que devem ser considerados durante o projeto da linguagem.

Uma vez que este livro também trata da implementação de linguagens de programação, este capítulo inclui uma visão geral das abordagens mais comuns à implementação. Por fim, descreveremos brevemente alguns exemplos de ambientes de programação e discutiremos seu impacto na produção de software.

1.1 Motivos para Estudar os Conceitos de Linguagens de Programação

É natural que os estudantes imaginem como se beneficiarão do estudo dos conceitos de linguagens de programação. Afinal de contas, uma grande quantidade de outros tópicos das ciências da computação merece um estudo sério. O que apresentamos a seguir é o que acreditamos ser uma lista obrigatória dos benefícios potenciais de estudar os conceitos da linguagem.

- *Aumento da capacidade de expressar idéias.* Acredita-se que a profundidade de nossa capacidade intelectual seja influenciada pelo poder expressivo da linguagem em que comunicamos nossos pensamentos. Os que possuem uma compreensão limitada da linguagem natural são limitados na complexidade de expressar seus pensamentos, especialmente em termos de profundidade de abstração. Em outras palavras, é difícil para as pessoas conceberem estruturas que não podem descrever, verbalmente ou por escrito. Programadores inscritos no processo de desenvolver software vêem-se similarmente embaraçados. A linguagem na qual desenvolvem o software impõe limites quanto aos tipos de estruturas de controle, de estruturas de dados e de abstrações que eles podem usar; assim, as formas de algoritmos possíveis de serem construídas também são limitadas.

O conhecimento de uma variedade mais ampla de recursos de linguagens de programação reduz essas limitações no desenvolvimento de software. Os programadores podem aumentar a variedade de seus processos intelectuais de desenvolvimento de software aprendendo novas construções de linguagem.

Pode-se argumentar que aprender as capacidades de outras linguagens não ajudará um programador obrigado a usar uma linguagem sem essas capacidades. Esse argumento não se sustenta, porém, porque freqüentemente as facilidades da linguagem podem ser simuladas em outras linguagens que não suportam esses recursos diretamente.

Por exemplo, depois de ter aprendido as funções de manipulação de *strings** do FORTRAN 90 (ANSI, 1992), como, por exemplo, a função de procura, INDEX, um programador Pascal (Ledgard, 1984) seria levado naturalmente a construir subprogramas para oferecer essas operações. O mesmo é verdadeiro em relação a muitas outras construções complexas discutidas neste livro.

O estudo dos conceitos das linguagens de programação forma uma apreciação dos recursos valiosos da linguagem e encoraja os programadores a usá-los.

O fato de muitos recursos das várias linguagens poderem ser simulados em outras não diminui significativamente a importância de projetar linguagens com o melhor conjunto de recursos. Sempre é melhor usar um recurso cujo projeto foi integrado na linguagem do que usar uma simulação desse, o qual, muitas vezes, é menos elegante e mais desajeitado em uma linguagem que não o suporta.

- *Maior conhecimento para a escolha de linguagens apropriadas.* Muitos programadores profissionais tiveram pouca educação formal em ciências da computação; ao contrário, aprenderam a programar sozinhos ou por meio de programas de treinamento *in-house***. Tais programas freqüentemente ensinam uma ou duas linguagens diretamente pertinentes ao trabalho da organização. Muitos outros programadores receberam seu treinamento formal em um passado distante. As linguagens que aprenderam não são mais usadas, e muitos recursos agora disponíveis não eram amplamente conhecidos. O resultado disso é que muitos programadores, quando lhes é dada a possibilidade de escolha das linguagens para um novo projeto, continuam a usar aquela com a qual estão mais familiarizados, mesmo que ela seja pouco adequada ao novo projeto. Se tais programadores estivessem mais familiarizados com as outras linguagens disponíveis, especialmente com os seus recursos particulares, estariam em uma posição melhor para fazerem uma escolha consciente.
- *Capacidade aumentada para aprender novas linguagens.* A programação de computadores é uma disciplina jovem, e as metodologias de projeto, as ferramentas de desenvolvimento de software e as linguagens de programação ainda estão em um estágio de contínua evolução. Isso torna o desenvolvimento de software uma profissão excitante, mas também significa que a aprendizagem contínua é fundamental. O processo de aprender uma nova linguagem de programação pode ser extenso e difícil, especialmente para alguém que esteja à vontade com somente uma ou com duas linguagens e que jamais examinou os conceitos em geral. Assim que for adquirida uma completa compreensão dos conceitos fundamentais das linguagens, irá tornar-se mais fácil ver como esses estão incorporados ao projeto da linguagem aprendida.

Por exemplo, programadores que entendem o conceito de abstração de dados terão mais facilidade para aprender como construir tipos de dados abstratos em Java (Gosling *et al.*, 1996) do que aqueles que não estão absolutamente familiarizados com tal exigência. O mesmo fenômeno ocorre nas linguagens naturais. Quanto mais você conhece a gramática de sua língua nativa, mais fácil achará aprender uma segunda língua natural. Além disso, aprender uma segunda língua

*N. de T. *String*: em programação, um conjunto contíguo qualquer de caracteres alfanuméricos. Nomes, endereços, palavras e frases são *strings*.

**N. de T. *In-house*: dentro da própria organização.

também tem o efeito colateral benéfico de ensiná-lo mais a respeito de seu primeiro idioma.

Por fim, é essencial que os programadores ativos conheçam o vocabulário e os conceitos fundamentais das linguagens de programação, para que possam ler e entender seus manuais e sua literatura de vendas de linguagens e de compiladores.

- *Entender melhor a importância da implementação.* Ao aprender os conceitos de linguagens de programação, tanto é interessante como necessário tocar nas questões de implementação que afetam esses conceitos. Em alguns casos, a compreensão das questões de implementação leva a um entendimento do porquê das linguagens serem projetadas daquela maneira. Isso, por sua vez, leva à capacidade de usar uma linguagem de modo mais inteligente, como ela foi projetada. Podemos nos tornar melhores programadores ao entendermos as escolhas que podemos fazer entre as construções de linguagens de programação e as consequências das opções.

Certos tipos de *bugs* de programa somente podem ser encontrados e corrigidos por um programador que conheça certos detalhes de implementação relacionados. Outro benefício de entender as questões de implementação é que isso nos permite visualizar como um computador executa várias construções da linguagem. Esta última afirmação, por sua vez, fomenta o entendimento da eficiência relativa de construções alternativas a serem escolhidas para o programa. Por exemplo, programadores que sabem pouco da implementação da recursão, muitas vezes, não sabem que um algoritmo recursivo é tipicamente muito mais lento do que um iterativo equivalente.

- *Aumento da capacidade de projetar novas linguagens.* Para um estudante, a exigência de um projeto de uma nova linguagem de programação no futuro pode parecer remota. Porém, a maioria dos programadores profissionais ocasionalmente projeta linguagens de um tipo ou de outro. Por exemplo, muitos dos sistemas exigem que o usuário interaja de alguma maneira, mesmo que somente para introduzir dados e comandos. Em situações simples, somente alguns valores de dados são introduzidos, e a linguagem do formato de entrada é trivial. Por outro lado, pode ser necessário que o usuário percorra diversos níveis de menus e introduza uma variedade de comandos, como no caso de um processador de texto. Nesses sistemas, a interface com o usuário é um problema de projeto complexo. A sua forma é projetada pelo desenvolvedor de sistemas, e os critérios para julgá-la são semelhantes aos usados para julgar o projeto de uma linguagem de programação. Um exame crítico das linguagens de programação, portanto, ajudará no projeto desses sistemas complexos e, mais comumente, ajudará os usuários a examinar e a avaliar esses produtos.
- *Avanço global da computação.* Por fim, há uma visão global da computação que pode justificar o estudo dos conceitos das linguagens de programação. Não obstante normalmente ser possível determinar o motivo pelo qual uma linguagem particular de programação tornou-se popular, nem sempre é claro, pelo menos em retrospectiva, que as linguagens mais populares são as melhores disponíveis. Em alguns casos, pode-se concluir que uma linguagem tornou-se popular porque aqueles com capacidade de optar ainda não estavam familiarizados com os conceitos de linguagem de programação.

Por exemplo, muitas pessoas acreditam que teria sido melhor se o ALGOL 60 (Backus *et al.*, 1962) tivesse substituído o FORTRAN no início da década de 60, porque aquela é mais elegante e tem instruções de controle muito melhores do

que este, entre outras razões. O fato disso não ter acontecido deve-se parcialmente aos programadores e aos gerentes de desenvolvimento de software daquela época, cuja maioria não entendia claramente o projeto conceitual do ALGOL 60. Eles achavam sua descrição difícil de ler (o que era verdade) e ainda mais difícil de entender. Não apreciaram os benefícios da estrutura em bloco, da recursão e das instruções de controle bem estruturadas, de modo que deixaram de ver os benefícios do ALGOL 60 sobre o FORTRAN.

Obviamente, muitos outros fatores contribuíram para a falta de aceitação do ALGOL 60, conforme veremos no Capítulo 2. Entretanto, o fato dos usuários de computador geralmente não terem consciência dos benefícios da linguagem desempenhou um papel importante.

Em geral, se aqueles que escolhem as linguagens forem melhor informados, talvez linguagens melhores se sobreporiam mais rapidamente às ruínas.

1.2 Domínios de Programação

Os computadores são usados em uma infinidade de diferentes áreas, desde o controle de usinas elétricas nucleares à armazenagem de registros de talões de cheques pessoais. Por causa dessa grande diversidade no seu espaço, linguagens de programação com metas muito diferentes têm sido desenvolvidas. Nesta seção, discutiremos brevemente algumas das áreas de aplicações de computadores e suas linguagens associadas.

1.2.1 Aplicações Científicas

Os primeiros computadores digitais, que surgiram na década de 40, eram usados e, de fato, foram inventados para aplicações científicas. Tipicamente, as aplicações científicas têm estruturas de dados simples, mas exigem um grande número de computações aritméticas com ponto-flutuante. As estruturas de dados mais comuns são os *arrays** e as matrizes (*matrices*); as estruturas de controle mais comuns são os laços de contagem e de seleções. As linguagens de programação de alto nível, inventadas para aplicações científicas, foram projetadas para suprir essas necessidades. A concorrente delas foi a linguagem *assembly*; desse modo, a eficiência era a primeira preocupação. A primeira linguagem para aplicações científicas foi o FORTRAN. O ALGOL 60 e a maioria de suas descendentes também se destinam a serem usadas nessa área, ainda que tenham sido projetadas também para outras áreas relacionadas. Para algumas aplicações científicas cuja eficiência é a principal preocupação, como

*N. de T. *Array*: (1) arranjo ordenado. Em computação, arranjo de elementos de memória em um ou em diversos níveis ou planos; matriz (coleção de dados similares armazenados sob o mesmo nome) ou estrutura ordenada de elementos acessíveis, referenciados por números, usada para conter tabelas ou conjunto de dados relacionados e do mesmo tipo. (2) (programação) Um conjunto de itens de dados de tipos idênticos, distinguidos por seus índices (ou "subscritos"). O número de dimensões que um *array* pode ter depende da linguagem, mas usualmente ele é ilimitado. Uma variável comum simples ("escalar") poderia ser considerada como um *array* zero-dimensional. Uma referência a um elemento de *array* é escrita como $A[i,j,k]$, em que A é o nome do *array* e i, j e k são os índices. A linguagem C é peculiar em termos de que cada índice é escrito em colchetes separados, e.g. $A[i][j][k]$. Isso expressa o fato de que, em C, um *array* N-dimensional é, de fato, um vetor, sendo que cada um de seus elementos é um *array* N-1 dimensional. Os elementos de um *array* são armazenados contiguamente.

aquelas comuns nas décadas de 50 e 60, nenhuma linguagem subsequente é significativamente melhor do que o FORTRAN.

1.2.2 Aplicações Comerciais

O uso de computadores para aplicações comerciais iniciou-se na década de 50. Equipamentos especiais foram desenvolvidos para tal propósito, juntamente com linguagens especiais. A primeira linguagem de alto nível bem-sucedida para negócios foi o COBOL (ANSI, 1985), que apareceu em 1960. Ela ainda é a mais comumente usada para essas aplicações. As linguagens comerciais são caracterizadas por facilidades para produzir relatórios elaborados, por maneiras precisas de descrever e por armazenar números decimais e dados de caracteres, além da capacidade de especificar operações aritméticas decimais.

Com o advento dos microcomputadores, surgiram novas maneiras de usar computadores para fazer negócios, especialmente de pequeno porte. Duas ferramentas específicas que podem ser usadas em pequenos computadores, os sistemas de planilhas eletrônicas e os sistemas de bancos de dados, foram desenvolvidas para os negócios e agora são amplamente usadas.

Há poucos desenvolvimentos nas linguagens de aplicação comercial além do COBOL. Portanto, este livro não discute as linguagens de aplicação comercial, a não ser para apresentar a história do desenvolvimento do COBOL no Capítulo 2.

1.2.3 Inteligência Artificial

A inteligência artificial (IA) é uma área abrangente das aplicações de computador caracterizada pelo uso de computações simbólicas em vez de numéricas. Computação simbólica significa que símbolos, que consistem em nomes no lugar de números, são manipulados. Além disso, a computação simbólica é feita de maneira mais conveniente com listas encadeadas de dados em vez de *arrays*. Esse tipo de programação, às vezes, requer mais flexibilidade do que outros domínios de programação. Por exemplo, em algumas aplicações de IA a capacidade de criar e de executar segmentos de código durante a execução é conveniente.

A primeira linguagem de programação desenvolvida para aplicações de IA amplamente utilizada foi a funcional LISP (McCarthy *et al.*, 1965), que surgiu em 1959. A maioria das aplicações de IA foi escrita em LISP ou em uma de suas parentes próximas. No início da década de 70, surgiu uma abordagem alternativa para tais aplicações — a programação lógica, usando a linguagem Prolog (Clocksin e Mellish, 1997). O Scheme, um dialeto do LISP, e o Prolog são apresentados nos Capítulos 14 e 15, respectivamente.

1.2.4 Programação de Sistemas

O sistema operacional e todas as ferramentas de suporte à programação de um computador são coletivamente conhecidos como seu **software básico** que é usado quase continuamente e, portanto, deve ter eficiência na execução. Portanto, uma linguagem para tal domínio deve oferecer uma execução rápida. Além disso, deve ter recursos de baixo nível que permitam ao software fazer interface com os dispositivos externos a serem escritos.

Nas décadas de 60 e 70, alguns fabricantes de computadores como, por exemplo, a IBM, a Digital e a Burroughs (agora UNISYS), desenvolveram linguagens de alto nível especiais orientadas para a máquina, ou seja, para o software de seus equipamentos. Para os

computadores *mainframe** da IBM, a linguagem era a PL/S, um dialeto da PL/I; para a Digital, era a BLISS, em uma linguagem num nível logo acima do *assembly*; para a Burroughs, era a Extended ALGOL.

O sistema operacional UNIX foi escrito quase inteiramente em C (ANSI, 1989), o que o tornou relativamente fácil de portar ou de mover, para máquinas diferentes. Algumas das características do C tornam boa a sua aplicação em programação de sistemas. Ela é de baixo nível, sua execução é eficiente e não sobrecarrega o usuário com muitas restrições de segurança. Os programadores de sistemas freqüentemente são excelentes e não acreditam precisar dessas restrições. Alguns, entretanto, acham a linguagem C muito perigosa para ser usada em sistemas grandes e importantes.

1.2.5 Linguagens de *Scripting*

As linguagens de *scripting* desenvolveram-se lentamente no decorrer dos últimos 25 anos. Tais linguagens são usadas colocando-se uma lista de comandos, chamados de *script*, em um arquivo para serem executados. A primeira, chamada de *sh* (de *shell*), iniciou-se como uma pequena coleção de comandos interpretados como chamadas aos subprogramas do sistema que executavam funções de utilidade como, por exemplo, gerenciamento e filtragem simples de arquivo. A essa base foram adicionadas variáveis, instruções de fluxo de controle, funções e várias outras capacidades. O resultado é uma linguagem de programação completa. Uma das mais poderosas e conhecidas delas é a *ksh* (Bolsky e Korn, 1995), desenvolvida por David Korn no *Bell Laboratories*.

A *awk* é outra linguagem de *scripting*, desenvolvida por Al Aho, Brian Kernighan e Peter Wienberger no *Bell Laboratories* (Aho et al., 1988). A *awk* começou como uma linguagem de geração de relatórios, mas, depois, passou a ser usada para propósitos mais gerais. A *tc1* é uma linguagem de *scripting* extensível, desenvolvida por John Ousterhout na Universidade da Califórnia em Berkeley (Ousterhout, 1994). A *tc1* agora está combinada com a *tk*, uma linguagem que fornece um método para construir aplicativos X Window. A linguagem Perl, desenvolvida por Larry Wall, era originalmente uma combinação da *sh* e da *awk* (Wall et al., 1996). A Perl desenvolveu-se significativamente desde seu início, e tornou-se uma linguagem de programação poderosa, ainda que um tanto primitiva. Não obstante ela ainda ser chamada freqüentemente de linguagem de *scripting*, preferimos imaginá-la como uma de programação estranha, mas completa. Desde o advento da World Wide Web, a popularidade da Perl cresceu drasticamente, principalmente porque ela é quase ideal para programação de *Common Gateway Interface* (CGI).

As linguagens de *scripting*, de maneira geral, têm contribuído pouco para o desenvolvimento de linguagens de programação mais convencionais. Porém, a Perl tem diversos recursos interessantes que discutiremos posteriormente neste livro.

1.2.6 Linguagens para Propósitos Especiais

Uma grande quantidade de linguagens para propósitos especiais surgiu no decorrer dos últimos 40 anos. Elas variam da RPG, usada para produzir relatórios comerciais, à APT, usada para instruir ferramentas de máquina programáveis, à GPSS, usada para simulação de sistemas. Este livro não discute as linguagens para propósitos especiais, principalmente

*N. de T. *Mainframe*: computador de grande porte.

por causa de sua estreita aplicabilidade e da dificuldade de compará-las com outras linguagens.

1.3 Critérios de Avaliação da Linguagem

Conforme observou-se, o propósito deste livro é examinar cuidadosamente os conceitos fundamentais das várias construções e das capacidades das linguagens de programação. Também avaliaremos esses recursos, concentrando-nos em seu impacto sobre o processo de desenvolvimento (inclusive manutenção) de software. Para levarmos isso a efeito, precisaremos de um conjunto de critérios de avaliação. Porém, uma lista de critérios é necessariamente controversa, porque é virtualmente impossível conseguir até mesmo dois cientistas da computação que concordem com o valor de determinada característica de linguagem em relação a outras. Apesar dessas diferenças, a maioria dos cientistas da computação concordaria que os critérios discutidos nas subseções seguintes são importantes.

Algumas das características que influenciam os mais importantes critérios são mostradas na Tabela 1.1; os mesmos são discutidos nas seções seguintes.

1.3.1 Legibilidade

Um dos critérios mais importantes para julgar uma linguagem de programação é a facilidade com que os programas podem ser lidos e entendidos. Antes de 1970, o desenvolvimento de software era muito imaginado em termos da escrita do código. Na década de 70, entretanto, o conceito de ciclo de vida do software (Booch, 1987) foi desenvolvido; a codificação foi relegada a um papel muito menos importante, e a manutenção foi reconhecida como uma parte importante do ciclo, especialmente em termos de custo. Uma vez que a facilita-

TABELA 1.1 Critérios de Avaliação da Linguagem e as Características que os Afetam

Característica	Critérios		
	Legibilidade	Capacidade de Escrita (Writability)	Confiabilidade
Simplicidade/ortogonalidade	•	•	•
Estruturas de controle	•	•	•
Tipos de dados e estruturas	•	•	•
Projeto da sintaxe	•	•	•
Suporte para abstração		•	•
Expressividade		•	•
Verificação de tipos			•
Manipulação de exceções			•
Aliasing* restrito			•

*N. de T. *Aliasing*: determinação de um nome alternativo ou de um apelido.

de de manutenção é determinada, em grande parte, pela legibilidade dos programas, ela tornou-se uma medida importante da qualidade dos programas e das linguagens.

A legibilidade deve ser considerada no contexto do domínio do problema. Por exemplo, se um programa que descreve uma computação tiver sido escrito em uma linguagem não-projetada para esse uso, o programa pode ser antinatural e enrolado, tornando-o incômodamente difícil de ser lido.

As subseções seguintes descrevem características que contribuem para a legibilidade de uma linguagem de programação.

1.3.1.1 Simplicidade Global

A simplicidade global de uma linguagem de programação afeta fortemente sua legibilidade. Antes de mais nada, uma linguagem com um grande número de componentes básicos é mais difícil de ser aprendida do que uma com poucos desses componentes. Os programadores que precisam usar uma linguagem grande tendem a aprender um subconjunto dela e ignorar seus outros recursos. Esse padrão de aprendizagem, às vezes, é usado para desviar-se do grande número de componentes da linguagem, mas tal argumento não é válido. Ocorrem problemas de legibilidade sempre que o autor do programa tenha aprendido um subconjunto diferente daquele com o qual o leitor está familiarizado.

Uma segunda característica que complica uma linguagem de programação é a multiplicidade de recursos — mais de uma maneira de realizar uma operação particular. Por exemplo, em C, o usuário pode incrementar uma variável inteira simples de quatro maneiras diferentes:

```
count = count + 1
count += 1
count++
++count
```

Não obstante as duas últimas instruções tenham significados ligeiramente diferentes entre elas e de todas as outras em alguns casos, todas as quatro têm o mesmo significado quando usadas como expressões independentes. Essas variações serão discutidas no Capítulo 6.

Um terceiro problema potencial é a sobrecarga* (*overloading*) de operador, na qual um único símbolo tem mais de um significado. Embora seja um recurso útil, pode levar a uma reduzida legibilidade se for permitido aos usuários criar suas próprias sobrecargas e não as constituírem criteriosamente. Por exemplo, é bem aceitável sobrecarregar `+` e usá-lo tanto para adição de números inteiros como para ponto-flutuante. Aliás, tal sobrecarga simplifica uma linguagem ao reduzir o número de operadores. Porém, suponhamos que o programador tenha definido que `+` seja usado entre operandos de *arrays* unidimensionais para significar a soma de todos os elementos de ambos os *arrays*. Uma vez que o significado da adição de vetores é bastante diferente deste último, isso tornaria o programa mais confuso tanto para o autor como para seus leitores. Um exemplo ainda mais extremo de confusão do programa seria um usuário definir `+` entre dois operandos de vetor para significar a diferença entre seus respectivos primeiros elementos. A sobrecarga de operadores será discutida adicionalmente no Capítulo 6.

A simplicidade nas linguagens, evidentemente, pode ser levada muito longe. Por exemplo, a forma e o significado da maioria das instruções da linguagem *assembly* são modelos

*N. de T. Sobrecarga: tradução literal de *overloading* que, em programação, é a possibilidade de usar o mesmo nome para mais de uma variável ou procedimento, exigindo que o compilador diferencie, baseando-se no contexto.

de simplicidade, como você poderá ver quando considerar as instruções que aparecem na próxima seção. Essa mesma simplicidade, entretanto, torna os programas em linguagem *assembly* menos legíveis. Uma vez que lhes faltam instruções de controle mais complexas, suas estruturas são menos evidentes; o fato de suas instruções serem simples exige um número bem maior do que o necessário para programas equivalentes escritos em uma linguagem de alto nível. Esses mesmos argumentos aplicam-se ao caso menos extremo das linguagens de alto nível com controle e com construções de estruturação de dados inadequados.

1.3.1.2 Ortogonalidade

Ortogonalidade em uma linguagem de programação significa que um conjunto relativamente pequeno de construções primitivas pode ser combinado em um número relativamente pequeno de maneiras para construir as estruturas de controle e de dados da linguagem. Além disso, toda combinação possível de primitivas é legal e significativa. Por exemplo, considere os tipos de dados. Suponhamos que uma linguagem tenha quatro tipos de dados primitivos, *integer*, *float*, *double* e *character*, e dois operadores de tipo, *array* e *pointer*. Se os dois operadores de tipo puderem ser aplicados a si mesmos e aos quatro tipos de dados primitivos, um grande número de estruturas de dados poderá ser definido. Porém, se não for permitido aos ponteiros (*pointers*) apontar para *arrays*, muitas dessas possibilidades seriam eliminadas.

O significado de um recurso de linguagem ortogonal é livre de contexto de sua aparência em um programa (o nome ortogonal vem do conceito matemático de vetores ortogonais, independentes um do outro). A ortogonalidade parte de uma simetria de relações entre primitivas. Os ponteiros devem ser capazes de apontar para qualquer tipo de variável ou estrutura de dados. A falta de ortogonalidade acarreta exceções às regras da linguagem.

Podemos ilustrar o uso da ortogonalidade como um conceito de projeto, comparando um aspecto das linguagens *assembly* dos computadores de grande porte da IBM com a série VAX de superminicomputadores. Consideramos uma única situação simples: adicionar valores inteiros de 32 bits que residem na memória ou nos registros e substituir um dos dois valores pela soma. Os computadores de grande porte da IBM têm duas instruções para essa finalidade sob as formas

```
A   Reg1, célula_de_memória
AR  Reg1, Reg2
```

em que Reg1 e Reg2 representam registros. A semântica delas é:

```
Reg1 ← conteúdo(Reg1) + conteúdo(célula_de_memória)
Reg1 ← conteúdo(Reg1) + conteúdo(Reg2)
```

A instrução de adição VAX para valores inteiros de 32 bits é

```
ADDL operando_1, operando_2
```

cuja semântica é

```
operando_2 ← conteúdo(operando_1) + conteúdo(operando_2)
```

Nesse caso, qualquer um dos operandos pode ser um registro ou uma célula de memória.

O projeto da instrução VAX é ortogonal pelo fato de uma única operação poder usar registros ou células de memória como operandos. São duas maneiras de especificar operandos que podem ser combinadas de todas as maneiras. O projeto IBM não é ortogonal. Somente duas combinações de operandos são legais em quatro possibilidades, e ambas exigem

diferentes instruções, **A** e **AR**. O projeto IBM é mais restrito e, portanto, menos fácil de ser escrito. Por exemplo, você não pode adicionar dois valores e armazenar a soma em uma localização da memória. Além disso, ele é mais difícil de ser aprendido por causa das restrições e da instrução adicional.

A ortogonalidade está estreitamente relacionada à simplicidade: quanto mais ortogonal é o projeto de uma linguagem, menos exceções as regras da linguagem exigirão. Menos exceções significam um grau mais elevado de regularidade no projeto, o que torna a linguagem mais fácil de ser aprendida, lida e entendida. Qualquer um que tenha aprendido uma parte significativa da língua inglesa pode atestar a dificuldade de entender suas muitas exceções à regra (por exemplo, *i* antes de *e*, exceto depois de *c*).

Como exemplos da falta de ortogonalidade em uma linguagem de alto nível, manifestada como exceções à regra, consideremos as seguintes regras em C. Não obstante a linguagem C possua dois tipos de dados estruturados, *arrays* e registros (*structs*), registros podem ser retornados de funções, mas *arrays* não. Um membro de uma estrutura pode ser qualquer tipo de dado, exceto *void* ou uma estrutura do mesmo tipo. Um elemento de *array* pode ser qualquer tipo de dado, exceto *void* ou uma função. Parâmetros são passados por valor, a menos que sejam *arrays*, em cujo caso são, com efeito, passados por referência (porque o aparecimento do nome de um *array* isoladamente, ou seja, sem nenhum colchete, em um programa em C é interpretado como o endereço do primeiro elemento do *array*). Uma expressão de adição simples, como

$a + b$

usualmente significa que os valores de *a* e *b* são pegos da memória e adicionados. Porém, se acontecer de *a* ser um ponteiro, o valor pego de *b* pode ser modificado antes que a adição se desenvolva. Por exemplo, se *a* apontar para um valor que tenha dois bytes de extensão, o valor de *b* será multiplicado por 2 antes que a adição desenvolva-se. O tipo de *a*, que é o contexto esquerdo de

$+ b$

forçará o valor de *b* a ser modificado antes que ele seja adicionado a *a*.

Muita ortogonalidade também pode causar problemas. Talvez a linguagem de programação mais ortogonal seja o ALGOL 68 (van Wijngaarden *et al.*, 1969). Toda construção de linguagem em ALGOL 68 tem um tipo, e não há nenhuma restrição para ele. Além disso, a maioria das construções produz valores. Essa liberdade de combinação permite construções extremamente complexas. Por exemplo, uma condicional pode aparecer como o lado esquerdo de uma atribuição, juntamente com declarações e com outras várias instruções, contanto que o resultado seja uma localização. Essa forma extrema de ortogonalidade acarreta uma complexidade desnecessária. Além disso, uma vez que as linguagens exigem um grande número de primitivas, um elevado grau de ortogonalidade resultará em uma explosão de combinações. Sendo assim, mesmo que as combinações sejam simples, seu elevado número leva à complexidade.

Portanto, simplicidade em uma linguagem é, pelo menos em parte, o resultado de uma combinação de um número relativamente pequeno de construções primitivas e o uso limitado do conceito de ortogonalidade.

Alguns acreditam que as linguagens funcionais oferecem uma boa combinação de simplicidade e de ortogonalidade. Uma linguagem funcional, como por exemplo o LISP, é uma linguagem em que as computações são feitas principalmente aplicando funções a determinados parâmetros. Em comparação, nas linguagens imperativas como C, Pascal e Java, as computações normalmente são especificadas com variáveis e com instruções de atribuição. As linguagens funcionais oferecem potencialmente a maior simplicidade global porque

podem realizar tudo com uma única construção, a chamada à função (*function call*), a qual pode ser combinada com outras chamadas a funções de maneira simples. Essa elegância simples é a razão pela qual alguns pesquisadores de linguagens são atraídos para as linguagens funcionais como a primeira alternativa para as linguagens não-funcionais complexas como o C++ (Ellis e Stroustrup, 1990). Outros fatores, como a eficiência, porém, têm impedido que as linguagens funcionais tornem-se mais populares.

1.3.1.3 Instruções de Controle

A revolução da programação estruturada da década de 70 foi, em parte, uma reação à má legibilidade causada pelas limitadas instruções de controle de algumas das linguagens das décadas de 50 e 60. Em particular, reconheceu-se amplamente que o uso indiscriminado das instruções *goto* reduz criticamente a legibilidade do programa.

Um programa que pode ser lido de cima a baixo é muito mais fácil de entender do que o que exige ao leitor pular de uma instrução a outra não-adjacente para seguir a ordem de execução. Em certas linguagens, entretanto, instruções *goto* que se ramificam para cima, às vezes, são necessárias; por exemplo, elas constroem laços* *WHILE* em FORTRAN 77. Restringir instruções *goto* das seguintes maneiras pode tornar os programas mais legíveis:

- Elas devem preceder seus alvos, exceto quando usadas para formar laços.
- Seus alvos nunca devem estar muito distantes.
- Seu número deve ser limitado.

Faltavam, às versões do BASIC e do FORTRAN disponíveis no início da década de 70, as instruções de controle que permitem fortes restrições ao uso de *gotos*, de modo que escrever programas altamente legíveis nessas linguagens era difícil. A maioria das linguagens de programação projetadas desde o final da década de 60, tem incluído instruções suficientes, de forma que a necessidade da instrução *goto* foi quase eliminada. Portanto, o projeto da estrutura de controle de uma linguagem agora é um fator menos importante na legibilidade do que no passado.

1.3.1.4 Tipos de Dados e Estruturas

A presença de facilidades adequadas para definir tipos de dados e estruturas de dados em uma linguagem é outro auxílio significativo para a legibilidade. Por exemplo, suponhamos que um tipo numérico seja usado para um sinalizador porque não há nenhum tipo booleano na linguagem. Nessa linguagem, poderíamos ter uma atribuição como

```
soma_e_muito_grande=1
```

cujo significado não é claro, enquanto que em uma linguagem com tipos booleanos, teríamos

```
soma_e_muito_grande=true
```

cujo significado é perfeitamente claro. Similarmente, tipos de dados registro (*record*) constituem um método para representar registros de empregados mais legível do que um conjunto de *arrays* similares, um para cada item de dados em um registro de empregado, o

*N. de T. Laço: *loop*. Em programação, uma repetição dentro de um programa. Sempre que um processo deve ser repetido, um laço é criado.

método exigido em uma linguagem sem registros. Por exemplo, no FORTRAN 77, uma matriz de registros de empregados poderia ser armazenada nos seguintes *arrays*:

```
CHARACTER (LEN = 30) NOME (100)
INTEGER IDADE (100), NUMERO_EMPREGADO (100)
REAL SALARIO (100)
```

Depois, um empregado particular é representado pelos elementos desses quatro *arrays* com o mesmo valor de índice.

1.3.1.5 Considerações sobre a Sintaxe

A sintaxe ou a forma dos elementos de uma linguagem têm um efeito significativo sobre a legibilidade dos programas. O que apresentamos a seguir são três exemplos de opções de projeto sintático que afetam a legibilidade:

- *Formas identificadoras.* Restringir os identificadores a tamanhos muito pequenos prejudica a legibilidade. Se os identificadores puderem ter seis caracteres no máximo, como no FORTRAN 77, muitas vezes não é possível usar nomes conotativos para as variáveis. Um exemplo mais extremo é o BASIC (ANSI, 1978b) do *American National Standards Institute* (ANSI), na qual um identificador poderia consistir somente de uma única letra ou de uma única letra seguida de um dígito.

Outras questões de projeto referentes a formas identificadoras serão discutidas no Capítulo 4.

- *Palavras especiais.* A aparência do programa e, desse modo, a sua legibilidade são fortemente influenciadas pelas formas das palavras especiais de uma linguagem (por exemplo, **begin**, **end** e **for**). Especialmente importante é o método para formar instruções compostas ou grupos de instrução principalmente em construções de controle. Diversas linguagens usam pares coincidentes de palavras ou de símbolos especiais para formar grupos. O Pascal exige pares **begin-end** para formar grupos em todas as construções de controle, exceto a instrução **repeat**, na qual elas podem ser omitidas (um exemplo da falta de ortogonalidade do Pascal). A linguagem C usa chaves para a mesma finalidade. Ambas as linguagens sofrem porque os grupos de instrução são sempre encerrados da mesma maneira, o que torna difícil determinar qual grupo está sendo finalizado quando um **end** ou **}** aparece. O FORTRAN 90 e a Ada tornam isso mais claro, usando uma sintaxe de fechamento distinta para cada tipo de grupo de instrução. Por exemplo, a Ada usa **end if** para finalizar uma construção de seleção, e **end loop** para finalizar uma construção de laço. Esse é um exemplo do conflito entre a simplicidade resultante de um número menor de palavras reservadas, como no Pascal, e a maior legibilidade que pode resultar do uso de um número maior de palavras reservadas, como na Ada.

Outra questão importante é se as palavras especiais de uma linguagem podem ser usadas como nomes para variáveis de programa. Se puderem, os programas resultantes podem ser muito confusos. Por exemplo, no FORTRAN 90, palavras especiais como **DO** e **END** são nomes de variáveis legais, de forma que o aparecimento de tais palavras em um programa pode conotar ou não algo especial.

- *Forma e significado.* Projetar instruções, a fim de que sua aparência indique, pelo menos, parcialmente sua finalidade, é um auxílio evidente para a legibilidade. A semântica, ou o significado, deve seguir diretamente da sintaxe ou da forma. Em alguns casos, esse princípio é violado por duas construções de linguagem idênti-

cas ou similares quanto à aparência, mas com significados diferentes, dependendo, talvez, do contexto. Em C, por exemplo, o significado da palavra reservada **static** depende do contexto de seu aparecimento. Se for usada na definição de uma variável dentro de uma instrução, significa que a variável é criada no momento da compilação (*compile time*). Se for usada na definição de uma variável fora de todas as funções, significa que esta é visível somente no arquivo em que sua definição aparece; ou seja, ela não é exportada desse arquivo.

Uma das principais reclamações a respeito dos comandos *shell* do UNIX (Kernighan e Pike, 1984) é que a aparência deles nem sempre sugere sua função. Por exemplo, o comando UNIX `grep` pode ser decifrado somente pelo conhecimento prévio, ou talvez pela habilidade e pela familiaridade com o editor UNIX, `ed`. Sua aparência não tem conotação alguma para os iniciantes UNIX. (No `ed`, o comando `/expressão_regular/` procura por uma *substring* que coincida com a expressão regular. Precedê-lo com `g` irá torná-lo um comando global, especificando que o escopo da procura é todo o arquivo que está sendo editado. Colocar um `p` depois do comando especificará que as linhas com *substrings* coincidentes sejam impressas. Assim, `g/expressão_regular/p`, que evidentemente pode ser abreviado para `grep`, imprime todas as linhas de um arquivo que contenham *substrings* que coincidem com a expressão regular.

1.3.2 Capacidade de Escrita (*Writability*)

Capacidade de escrita é uma medida de quão facilmente uma linguagem pode ser usada para criar programas para um domínio de problema escolhido. A maioria das características da linguagem que afeta a legibilidade também afeta a capacidade de escrita. Isso se segue diretamente do fato de que escrever um programa exige uma releitura freqüente da parte que já foi escrita pelo programador.

Como acontece com a legibilidade, a capacidade de escrita deve ser considerada no contexto do domínio de problema-alvo de uma linguagem. Simplesmente, não é razoável comparar a capacidade de escrita de duas linguagens no domínio de uma aplicação particular quando uma foi projetada para essa aplicação e a outra não. Por exemplo, a capacidade de escrita do COBOL (ANSI, 1985) e a da APL (Gilman e Rose, 1976) são drasticamente diferentes para criar um programa capaz de lidar com estruturas de dados bidimensionais, para as quais a APL é ideal. Suas capacidades de escrita também são diferentes na produção de relatórios financeiros com formatos complexos, para os quais o COBOL foi projetado.

As subseções seguintes descrevem os fatores mais importantes a influenciar a capacidade de escrita de uma linguagem.

1.3.2.1 Simplicidade e Ortogonalidade

Se uma linguagem tiver um grande número de diferentes construções, alguns programadores podem não estar familiarizados com todas elas. Isso pode levar ao erro inadequado de alguns recursos e ao desuso de outros que podem ser ou mais elegantes ou mais eficientes (ou ambos) do que aqueles usados. Pode, até mesmo, ser possível, como foi observado por Hoare (1973), usar recursos desconhecidos acidentalmente, com resultados bizarros. Portanto, um número menor de construções primitivas e um conjunto consistente de regras para combiná-las (isto é, ortogonalidade) é muito melhor do que, simplesmente, ter um número grande de primitivas. Um programador pode projetar uma solução para um problema complexo depois de aprender somente um conjunto simples de construções primitivas.

Por outro lado, demasiada ortogonalidade pode resultar em prejuízo para a capacidade de escrita. Erros ao escrever programas podem não ser detectados, uma vez que quase todas as combinações de primitivas são legais. Isso pode levar a absurdos no código que não podem ser descobertos pelo compilador.

1.3.2.2 Suporte para Abstração

Colocando brevemente, **abstração** significa a capacidade de definir e, depois, de usar estruturas ou operações complicadas de uma maneira que permita ignorar muitos dos detalhes. A abstração é um conceito fundamental no projeto de linguagens de programação contemporâneas. Isso é um reflexo do papel central que a abstração desempenha nas modernas metodologias de projeto de programas. O grau de abstração permitido por uma linguagem de programação e a naturalidade de sua expressão são, por conseguinte, muito importantes para sua capacidade de escrita (*writability*). As linguagens de programação podem suportar duas categorias distintas de abstração: processo e dados.

Um exemplo simples de abstração do processo é o uso de um subprograma para implementar um algoritmo de classificação exigido diversas vezes em um programa. Sem este último, o código de classificação teria de ser replicado em todos os lugares em que fosse necessário; isso tornaria o programa muito mais longo e mais tedioso de ser escrito. O mais importante, se o subprograma não fosse usado, é que o código que usou o subprograma de classificação seria atravancado com detalhes do algoritmo de classificação, obscurecendo grandemente o fluxo e o intento global desse código.

Como um exemplo de abstração de dados, considere uma árvore binária que armazena dados inteiros em seus vértices e que seria normalmente implementada em FORTRAN 77 como três *arrays* inteiros paralelos, em que dois dos inteiros (*integers*) são usados como subscritos* para especificar os vértices descendentes. Em C++ e em Java, esses três podem ser implementados usando-se uma abstração de um vértice da árvore na forma de uma classe simples com dois ponteiros e um número inteiro. A naturalidade desta última representação torna muito mais fácil de escrever um programa com árvores binárias em tais linguagens do que escrever um em FORTRAN 77. É uma simples questão do domínio de solução de problemas da linguagem estar mais próxima do domínio do problema.

O suporte global para abstração, evidentemente, é um fator importante na capacidade de escrita de uma linguagem.

1.3.2.3 Expressividade

Expressividade, em uma linguagem, pode referir-se a diversas características diferentes. Na linguagem APL, por exemplo, significa que há operadores muito poderosos permitindo que uma grande quantidade de computação seja realizada com um programa muito pequeno. Mais comumente, significa que uma linguagem tem formas relativamente convenientes, em vez de desajeitadas, de especificar computações. Por exemplo, em C, a notação `count++` é mais conveniente e mais breve do que `count = count + 1`. Também, o operador booleano **and then** em Ada é uma maneira conveniente de especificar uma avaliação em curto-circuito de uma expressão booleana. A inclusão da instrução **for** em Pascal torna mais fácil a escrita de laços de contagem do que com o uso de **while**, também possível. Todos eles aumentam a capacidade de escrita de uma linguagem.

*N. de T. Subscrito: *subscript*. É um método para referenciar dados em uma tabela. Por exemplo, na tabela Tabpreco, a instrução para referenciar um preço específico pode ser `tabpreco (item)`, onde `item` é variável de subscrito.

1.3.3 Confiabilidade

Diz-se que um programa é **confiável** se ele se comporta de acordo com suas especificações sob todas as condições. As subseções seguintes descrevem diversos recursos de linguagem que exercem um efeito significativo sobre a confiabilidade de programas.

1.3.3.1 Verificação de Tipos

Verificar tipos é, simplesmente, testar se existem erros de tipo em determinado programa, ou por meio do compilador ou durante a execução do programa. A verificação de tipos é um fator importante na confiabilidade da linguagem. Uma vez que a verificação de tipos em tempo de execução (*run-time*) é dispendiosa, a verificação em tempo de compilação é mais desejável. Além disso, quanto mais cedo forem detectados erros em um programa, menos dispendioso será para fazer os reparos necessários. O projeto da Ada exige verificações dos tipos de quase todas as variáveis e expressões em tempo de compilação, exceto quando o usuário declara explicitamente que a verificação de tipos deve ser suspensa. Isso virtualmente elimina os erros de tipo durante a execução de programas escritos em Ada. Os tipos e a verificação destes serão descritos com profundidade nos Capítulos 4 e 5.

Um exemplo de como a falta de verificação de tipos, em tempo de compilação ou em tempo de execução, levam a incontáveis erros de programa é o uso de parâmetros de subprograma na linguagem C original (Kernighan e Ritchie, 1978). Nessa linguagem, o tipo de um parâmetro real em uma chamada a função não é verificado para determinar se seu tipo coincide com o do parâmetro formal correspondente na função. Uma variável do tipo **int** pode ser usada como um parâmetro real em uma chamada a uma função que espera um tipo **float** como seu parâmetro formal, e nem o compilador, nem o sistema em tempo de execução detectarão a incoerência. Isso acarreta problemas, cuja fonte, muitas vezes, é difícil de determinar-se (em resposta a esse e a outros problemas semelhantes, os sistemas UNIX incluem um programa utilitário chamado `lint` que verifica se existem problemas em programas C). Os subprogramas e a passagem de parâmetros serão discutidos no Capítulo 8.

Em Pascal, a faixa de subscrito de uma variável *array* faz parte do tipo da variável. Portanto, a verificação da faixa de subscrito faz parte da verificação de tipos, não obstante ela deva ser feita em tempo de execução (*run-time*). Uma vez que a maioria dos tipos é verificada no Pascal, as faixas de subscrito também são testadas. Isso é extremamente importante para a confiabilidade do programa, porque os subscritos fora da faixa frequentemente causam erros que não aparecem até muito tempo depois de acontecer violações reais. As linguagens Ada e Java também exigem que todos os subscritos sejam verificados quanto à faixa.

1.3.3.2 Manipulação de Exceções

A capacidade de um programa de interceptar erros em tempo de execução (bem como outras condições incomuns detectadas pelo programa), pôr em prática medidas corretivas e, depois, prosseguir é um grande auxílio para a confiabilidade. Tal facilidade da linguagem é chamada de **manipulação de exceções**^{*}. Ada, C++ e Java incluem grandes capacidades de manipular exceções, mas essas facilidades praticamente não existem em muitas lin-

^{*}N. de T. Manipulação de exceções: também chamada de tratamento de exceções.

guagens mais populares, como o C e o FORTRAN. A manipulação de exceções será discutida no Capítulo 13.

1.3.3.3 *Aliasing*

Definido livremente, ***aliasing*** é ter dois ou mais métodos, ou nomes, distintos para fazer referência à mesma célula da memória. Agora é amplamente aceito que o *aliasing* seja um recurso perigoso em uma linguagem de programação. A maioria das linguagens de programação permite algum tipo de *aliasing* — por exemplo, membros de união e ponteiros definidos para apontar para a mesma variável em C. Em ambos os casos, duas diferentes variáveis de programa podem referir-se à mesma célula da memória. Alguns tipos de *aliasing*, descritos nos Capítulos 4 e 8, podem ser proibidos pelo projeto de uma linguagem.

Em algumas linguagens, o *aliasing* é usado para superar deficiências nas facilidades de abstração de dados. Outras linguagens o restringem muito para aumentarem sua confiabilidade.

1.3.3.4 Legibilidade e Capacidade de Escrita

Tanto a legibilidade como a capacidade de escrita influenciam a confiabilidade. Um programa escrito em uma linguagem que não suporta maneiras naturais de expressar os algoritmos exigidos usará, necessariamente, métodos não-naturais. Estes últimos têm menos probabilidade de estarem corretos para todas as situações possíveis. Quanto mais fácil é escrever um programa, mais probabilidade ele tem de ser correto.

A legibilidade afeta a confiabilidade tanto nas fases de escrita como nas de manutenção no ciclo de vida. Programas de difícil leitura complicam também sua escrita e sua modificação.

1.3.4 Custo

O custo final de uma linguagem de programação é uma função de muitas de suas características.

Primeiro, há o custo do treinamento de programadores para usar a linguagem. Essa é uma função da simplicidade e da ortogonalidade da linguagem e da experiência dos programadores. Embora linguagens mais poderosas não sejam, necessariamente, mais difíceis de aprender, elas freqüentemente o são.

Em segundo lugar, há o custo para escrever programas na linguagem. Essa é uma função da capacidade de escrita, a qual depende de sua intensidade de propósito com a aplicação particular. Os esforços originais para projetar e implementar linguagens de alto nível foram motivados pelo desejo de diminuir os custos para criar software.

Tanto o custo para treinar programadores como para escrever programas em uma linguagem podem ser significativamente reduzidos em um bom ambiente de programação. Os ambientes de programação serão discutidos na Seção 1.7.

Em terceiro lugar, há o custo para compilar programas na linguagem. Um grande empecilho para os primeiros usos da Ada era o custo proibitivamente elevado para rodar os compiladores Ada de primeira geração. Esse problema foi diminuído pelo surgimento de compiladores melhores.

Em quarto lugar, o custo para executar programas é grandemente influenciado pelo projeto da linguagem. Se ela exigir muitas verificações de tipos durante a execução, proibirá a execução rápida de código, independentemente da qualidade do compilador. Ainda

que a eficiência de execução fosse a principal preocupação no projeto das primeiras linguagens, considera-se isso menos importante agora.

Pode ser feito um *trade-off* simples entre custo de compilação e velocidade de execução do código compilado. Otimização é o nome dado para a coleção de métodos que os compiladores podem usar para diminuir o tamanho e/ou aumentar a velocidade de execução do código produzido. Se ocorrer pouca ou nenhuma otimização, a compilação pode ser feita muito mais rapidamente do que se houver um esforço significativo para produzir código otimizado. O esforço de compilação extra resulta em uma execução de código mais rápida. A escolha entre as duas alternativas é determinada pelo ambiente no qual o compilador será usado. Em um laboratório para estudantes de programação iniciantes que usam um bocado de tempo de compilação, mas pouco de execução de código (seus programas são pequenos e eles devem executar corretamente somente uma vez), pouca ou nenhuma otimização deve ser feita. Em um ambiente de programação, em que programas completos são executados muitas vezes, é melhor pagar o custo extra para otimizar o código.

O quinto fator é o custo do sistema de implementação da linguagem. Um dos fatores que explicam a rápida aceitação de Java é que sistemas compiladores/interpretadores estavam à disposição para ela logo depois que seu projeto foi lançado pela primeira vez. Uma linguagem de programação cujo sistema de implementação seja caro, ou rode somente em hardware caro, terá muito menos chance de tornar-se popular.

O sexto fator é o custo da má confiabilidade. Se o software falhar em um sistema crítico, como uma usina de energia nuclear ou uma máquina de raios X, o custo poderia ser muito elevado. As falhas de sistemas não-críticos também podem ser muito caras em termos de futuro comercial ou de ações judiciais em função de sistemas de software defeituosos.

A consideração final é o custo de manutenção dos programas, que inclui tanto correções como modificações para adicionar novas capacidades. O custo da manutenção de software depende de uma série de características da linguagem, mas principalmente da legibilidade. Uma vez que a manutenção freqüentemente é feita por pessoas que não o autor original do software, uma legibilidade ruim pode tornar a tarefa extremamente desafiadora.

A importância da manutenção de software não pode ser exagerada. Estima-se que, para grandes sistemas de software com tempos de vida relativamente longos, os custos de manutenção podem atingir de duas a quatro vezes os custos de desenvolvimento (Sommerville, 1992).

De todos os fatores que contribuem para os custos da linguagem, três são os mais importantes: desenvolvimento do programa, manutenção e confiabilidade. Uma vez que esses são funções da capacidade de escrita e da legibilidade, os dois últimos critérios de avaliação são, por sua vez, os mais importantes.

Obviamente, um grande número de critérios está disponível para avaliar linguagens de programação. Um, por exemplo, é a portabilidade ou a facilidade com que os programas podem ser mudados de uma implementação para outra. A portabilidade é mais fortemente influenciada pelo grau de padronização da linguagem. Algumas linguagens, como o BASIC, não são padronizadas, tornando os programas muito difíceis de serem mudados de uma implementação para outra. A padronização é um processo difícil que consome tempo. Uma comissão começou a trabalhar para produzir uma versão padrão do C++ em 1989. Até o início de 1998, esse padrão ainda não havia sido concluído.

Generalidade (a aplicabilidade a uma ampla faixa de utilizações) e boa definição (*definedness*) (a perfeição e a precisão do documento oficial que define a linguagem) são dois outros critérios.

A maioria dos critérios, especialmente a legibilidade, a capacidade de escrita e a confiabilidade não é nem precisamente definida, nem exatamente mensurável. Eles são conceitos úteis, entretanto, e fornecem valiosas avaliações sobre o projeto e sobre as linguagens de programação.

Uma nota final sobre os critérios de avaliação: os critérios de projeto da linguagem são pesados diferentemente a partir de perspectivas diversas. Os implementadores da linguagem estão preocupados primeiramente com a dificuldade de implementar as construções e os recursos daquela. Os usuários da linguagem estão preocupados em primeiro lugar com a capacidade de escrita e depois com a legibilidade. Os projetistas provavelmente enfatizarão a elegância e a capacidade de atrair um uso generalizado. Essas características, às vezes, entram em conflito.

1.4 Influências sobre o Projeto da Linguagem

Além daqueles fatores descritos na Seção 1.3, vários outros influenciam no projeto básico das linguagens de programação. Os mais importantes são a arquitetura do computador e as metodologias de projeto do programa.

1.4.1 Arquitetura do Computador

A arquitetura básica dos computadores exerceu um efeito crucial sobre o projeto das linguagens. A maioria das mais populares dos últimos 35 anos foi projetada em função da arquitetura de computador prevalecente, chamada de arquitetura von Neumann, em homenagem a um de seus criadores, John von Neumann (pronuncia-se “fon Nóíman”). Essas linguagens são chamadas de *imperativas*. Em um computador de von Neumann, tanto os dados como os programas são armazenados na mesma memória. A unidade central de processamento (CPU), que executa realmente as instruções, é separada da memória. Portanto, as instruções e os dados devem ser canalizados (*piped*) ou transmitidos da memória para a CPU. Os resultados das operações na CPU devem ser novamente transferidos para a memória. Quase todos os computadores digitais construídos desde a década de 40 têm se baseado na arquitetura von Neumann. A estrutura global de um computador de von Neumann é mostrada na Figura 1.1 (ver p. 36).

Por causa da arquitetura von Neumann, os recursos centrais das linguagens imperativas são as variáveis, as quais modelam as células de memória, as instruções de atribuição, baseadas na operação de canalização (*piping*) e a forma iterativa de repetição, o método mais eficiente dessa arquitetura. Os operandos das expressões são canalizados da memória para a CPU, e o resultado da avaliação daquela é canalizado de volta para a célula da memória representada pelo lado esquerdo da atribuição. A iteração é rápida nos computadores de von Neumann porque as instruções são armazenadas em células adjacentes da memória. Essa eficiência desencoraja o uso da recursão para repetição, não obstante frequentemente ser mais natural.

Conforme foi declarado anteriormente, uma linguagem funcional ou aplicativa é aquela cujo principal meio de fazer computações é aplicando funções a determinados parâmetros. A programação pode ser feita em uma linguagem funcional sem o tipo de variáveis usadas nas imperativas, sem instruções de atribuição e sem iteração. Ainda que muitos cientistas

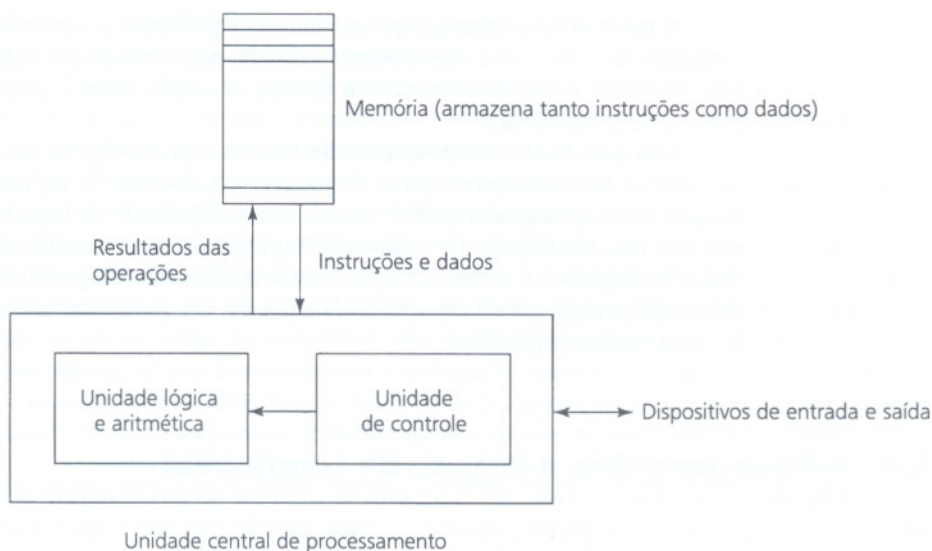


FIGURA 1.1 A arquitetura do computador de von Neumann.

da computação tenham feito exposições sobre os inúmeros benefícios das linguagens funcionais, como o LISP, é improvável que eles deixem de lado as imperativas até que um computador não-von Neumann seja projetado, permitindo a execução eficiente de programas em linguagens funcionais. Entre aqueles que lastimam tal fato, o mais eloquente tem sido John Backus, o projetista-chefe da versão original do FORTRAN (Backus, 1978).

As máquinas de arquitetura paralela que surgiram nos últimos 15 anos apresentam alguma promessa de agilizar a velocidade de execução de programas funcionais, mas, até agora, isso não tem sido suficiente para torná-las competitivas com programas imperativos. Aliás, ainda que existam maneiras elegantes de usar arquiteturas paralelas para executar programas funcionais, a maioria dessas é usada para programas imperativos, especialmente os escritos em dialetos do FORTRAN.

1.4.2 Metodologias de Programação

O final da década de 60 e o início da década de 70 trouxeram uma análise intensa, em grande parte iniciada pelo movimento de programação estruturada, tanto do processo de desenvolvimento de software como do projeto de linguagens de programação.

Uma razão importante para essa pesquisa foi a mudança no importante custo de computação do hardware para o software, uma vez que os custos daquele diminuíram e os custos dos programadores elevaram-se. Os aumentos de produtividade do programador foram relativamente pequenos. Além disso, problemas progressivamente maiores e mais complexos passaram a ser resolvidos por computador. Em vez de simplesmente resolver

conjuntos de equações para simular rastreamento de satélites, como no início da década de 60, passaram a ser feitos programas para tarefas grandes e complexas, como, por exemplo, controlar grandes instalações de refinaria de petróleo e oferecer sistemas de reservas de passagens aéreas no mundo inteiro.

As novas metodologias de desenvolvimento de software surgiram em consequência das pesquisas da década de 70 e foram chamadas de projeto *top-down* e refinamento passo a passo. As principais deficiências descobertas nas linguagens de programação foram a não-plenitude de verificação de tipos e a insuficiência das instruções de controle (que exigiam o uso excessivo de *gotos*).

No final década de 70, iniciou-se uma mudança das metodologias de projeto de programas orientadas para o processo para as orientadas a dados. Colocando de maneira simples, os métodos orientados a dados enfatizam o projeto de dados, concentrando-se no uso de tipos de dados abstratos para resolver problemas.

Para que a abstração de dados seja usada eficientemente no projeto de sistemas de software, ela deve ser suportada pelas linguagens voltadas para implementação. A primeira a oferecer um limitado suporte para abstração de dados foi a SIMULA 67 (Birtwistle *et al.*, 1973), não obstante ela certamente não ter sido levada à popularidade por causa disso. Os benefícios da abstração de dados não foram amplamente reconhecidos até o início da década de 70. Porém, a maioria das linguagens projetadas desde o final daquela década suporta abstração de dados que será discutida detalhadamente no Capítulo 10.

Os passos mais recentes na evolução do desenvolvimento de software orientado a dados, que se iniciou em meados da década de 80, é o projeto orientado a objeto. A metodologia orientada a objeto inicia-se com a abstração de dados, a qual encapsula o processamento com objetos de dados e oculta o acesso a eles, e adiciona herança e vinculação dinâmica de tipos. Herança é um conceito poderoso que aumenta muito a reutilização potencial dos softwares existentes, oferecendo, portanto, a possibilidade de significativos aumentos na produtividade de desenvolvimento de software. Esse é um fator importante no aumento de popularidade das linguagens orientadas a objeto. A vinculação dinâmica (sem tempo de execução) de tipos permite um uso mais flexível da herança.

A programação orientada a objeto desenvolveu-se juntamente com uma linguagem que suportou seus conceitos: a Smalltalk (Goldberg e Robson, 1983). Embora a Smalltalk não tenha sido tão usada como algumas outras linguagens, o suporte para programação orientada a objeto, agora, faz parte das linguagens de programação mais populares, inclusive a Ada 95 (AARM, 1995), o Java e o C++. Os conceitos orientados a objeto também chegaram à programação funcional em CLOS (Bobrow *et al.*, 1988) e programação lógica em Prolog++. O suporte à linguagem para programação orientada a objeto será discutido detalhadamente no Capítulo 11.

A programação orientada ao processo é, em certo sentido, o oposto da programação orientada a dados. Não obstante os métodos orientados a dados agora dominarem o desenvolvimento de software, os métodos orientados ao processo não foram abandonados. Ao contrário, tem ocorrido muita pesquisa em programação orientada a objeto nos últimos anos, especialmente na área da concorrência. Esses esforços de pesquisa trouxeram consigo a necessidade de facilidades de linguagem para criar e para controlar unidades de programa concorrentes. A Ada e o Java incluem essas capacidades. Desse modo, a evolução da programação exige novamente novas capacidades de linguagem. A concorrência será discutida detalhadamente no Capítulo 12.

1.5 Categorias de Linguagem

As linguagens de programação, muitas vezes, são categorizadas em quatro caixas: imperativas, funcionais, lógicas e orientadas a objeto. Já discutimos as características das linguagens imperativas e funcionais. Também descrevemos como as linguagens de programação orientadas a objeto mais populares desenvolveram-se a partir das imperativas. Não obstante o paradigma de desenvolvimento de software diferir bastante do paradigma orientado para procedimentos normalmente usado com as linguagens imperativas, as extensões a uma linguagem imperativa necessárias para suportar a programação orientada a objeto não são opressivas. Por exemplo, as expressões, as instruções de atribuição e as instruções de controle do C e do Java são quase idênticas; por outro lado, os *arrays*, os subprogramas e a semântica da linguagem Java são muito diferentes em C.

Uma linguagem de programação lógica é um exemplo baseado em regras. Em uma linguagem imperativa, um algoritmo é especificado com grandes detalhes, e a ordem de execução específica das instruções ou dos comandos deve ser incluída. Em uma linguagem baseada em regras, estas são especificadas sem nenhuma ordem particular, e o sistema de implementação deve escolher uma ordem de execução que produza o resultado desejado. Essa abordagem ao desenvolvimento de software é radicalmente diferente daquelas usadas com os outros três tipos de linguagens, e, evidentemente, exige um tipo de linguagem completamente diferente. O Prolog, a mais popular linguagem de programação lógica usada, e a programação lógica serão discutidas no Capítulo 15.

As linguagens de marcação* (*markup*), como a HTML, às vezes, são confundidas com as de programação. Porém, as linguagens de marcação não especificam computações; ao contrário, elas descrevem a aparência geral de documentos. Porém, muitos dos critérios de projeto e de avaliação descritos neste capítulo também se aplicam às linguagens de marcação. Afinal de contas, é evidentemente importante que o código de marcação seja fácil de escrever e de ler.

1.6 Trade-Offs no Projeto da Linguagem

Os critérios de avaliação das linguagens de programação descritos na Seção 1.3 fornecem uma estrutura para o projeto. Infelizmente, essa estrutura é contraditória. Em seu documento sobre projeto de linguagens, Hoare (1973) afirma que “existem tantos critérios importantes, mas conflitantes, que a reconciliação e a satisfação dos mesmos é uma grande tarefa de engenharia”.

Dois critérios conflitantes são a confiabilidade e o custo de execução. Por exemplo, a definição da linguagem Ada exige que todas as referências aos elementos do *array* sejam verificados para assegurar que o índice ou os índices estejam em suas faixas legais. Isso aumenta o custo de execução de programas Ada que contenham um grande número de referências aos elementos do *array*. O C não exige verificação da faixa de índice, de modo que os programas em C executam mais rapidamente do que programas em Ada semanticamente equivalentes, não obstante os programas em Ada sejam mais confiáveis. Os projetistas da Ada trocaram a eficiência de execução pela confiabilidade.

*N. de T. Marcação: *markup*. Um padrão para a definição de vínculos de hipertexto entre documentos.

Como outro exemplo de critérios conflitantes que levam diretamente a *trade-offs* no projeto, consideremos o caso da APL. Esta inclui um poderoso conjunto de operadores para operações com *array*. Por causa do grande número de operadores, um número significativo de novos símbolos teve de ser incluído na APL para representá-los. Além disso, muitos operadores APL podem ser usados em uma única expressão longa e complexa. Um resultado desse elevado grau de expressividade é que, para aplicações que envolvem muitas operações com *array*, a APL é muito fácil de ser escrita (*writable*). De fato, uma enorme quantidade de computação pode ser especificada em um programa muito compacto. Outro resultado é que os programas em APL têm uma legibilidade muito ruim. Uma expressão compacta e concisa tem certa beleza matemática, mas é difícil para qualquer outra pessoa entender, a não ser para quem a escreveu. O conhecido autor Daniel McCracken observou certa vez que demorava quatro horas para ler um programa APL de quatro linhas (McCracken, 1970). Os projetistas da APL trocaram a legibilidade por capacidade de escrita (*writability*).

O conflito entre flexibilidade e segurança é comum no projeto de uma linguagem. Os registros variantes em Pascal (*variant records*) permitem que uma célula de memória contenha diferentes valores de tipo em diferentes momentos. Por exemplo, uma célula pode conter ou um ponteiro, ou um número inteiro. Então, pode-se fazer uma operação sobre um valor de ponteiro colocado nesse tipo de célula como se ele fosse um número inteiro, usando-se qualquer operação definida para valores inteiros. Isso fornece uma saída na verificação de tipos da linguagem Pascal, que permite que um programa faça operações aritméticas nos ponteiros, o que, às vezes, é conveniente. Porém, esse uso não-verificado das células de memória é, em geral, uma prática perigosa.

Abundam exemplos de conflitos entre os critérios de projeto (e de avaliação) de linguagens; alguns são sutis, outros são óbvios. Portanto, é evidente que a tarefa de escolher construções e recursos ao projetar uma linguagem de programação envolve uma coleção de meios-termos e de *trade-offs*.

1.7 Métodos de Implementação

Como foi descrito na Seção 1.4.1, dois dos principais componentes de um computador são sua memória interna e seu processador. A memória interna é usada para armazenar programas e dados. O processador é um conjunto de circuitos que garante a realização de um conjunto de operações primitivas, ou de instruções de máquina, como aquelas para operações aritméticas e lógicas. Na maioria dos computadores, algumas dessas instruções, que, às vezes, são chamadas de macroinstruções, são de fato implementadas com um conjunto de instruções de nível ainda mais baixo, chamadas microinstruções. Uma vez que as microinstruções nunca são vistas pelo software ou pelos programadores, usualmente elas não são incluídas em qualquer discussão sobre softwares. Portanto, elas não serão discutidas adicionalmente aqui.

A linguagem de máquina do computador é seu conjunto de macroinstruções. Na ausência de outro software de suporte, sua própria linguagem de máquina é a única que a maioria dos computadores “entendem”. Teoricamente, um computador pode ser projetado e construído com uma linguagem de alto nível particular como sua linguagem de máquina, mas seria muito complexo e caro. Além disso, ele seria altamente inflexível, porque seria difícil (ainda que não impossível) de usar com outras linguagens de alto nível. A opção de projeto de máquina mais prática implementa em hardware uma linguagem de nível muito

baixo que oferece as operações primitivas mais comumente necessárias e exige que o software de sistema crie uma interface com os programas de nível mais elevado.

Um sistema de implementação de linguagem não pode ser o único software em um computador. Também é necessário um grande conjunto de programas, chamado sistema operacional, o qual fornece primitivas de nível mais alto do que as da linguagem de máquina. Essas primitivas oferecem gerenciamento de recursos do sistema, operações de entrada e saída, um sistema de gerenciamento de arquivos, editores de texto e/ou programas, e uma variedade de outras funções comumente necessárias. Uma vez que os sistemas de implementação de linguagem precisam de muitas das facilidades do sistema operacional, eles comunicam-se com o sistema operacional em vez de diretamente com o processador (na linguagem de máquina).

O sistema operacional e as implementações são dispostos em camadas sobre a interface da linguagem de máquina de um computador. Essas camadas podem ser imaginadas como computadores virtuais, que oferecem interfaces para o usuário em níveis mais altos. Por exemplo, um sistema operacional e um compilador C constituem um computador virtual C. Com outros compiladores, uma máquina pode transformar-se em outros tipos de computadores virtuais. A maioria dos computadores fornece diversas máquinas virtuais diferentes. Os programas de usuário formam outra camada no topo da camada das máquinas virtuais.

A visualização em camadas de um computador é mostrada na Figura 1.2 (ver p. 41).

Os sistemas de implementação das primeiras linguagens de programação de alto nível, construídos no final da década de 50, estavam entre os sistemas mais complexos daquela época. Na década de 60, esforços de pesquisa intensivos foram feitos para compreender e para formalizar o processo de construção dessas implementações de linguagem de alto nível. O maior sucesso desses esforços ocorreu na área de análise sintática, principalmente porque essa parte do processo de implementação é uma aplicação de partes da teoria dos autômatos* e da teoria da linguagem formal que eram, então, bem-entendidas.

1.7.1 Compilação

Linguagens de programação podem ser implementadas por meio de qualquer um dos três métodos gerais. Em um extremo, programas podem ser traduzidos para linguagem de máquina, a qual pode ser executada diretamente no computador. Isso é chamado de implementação **compilada**. Esse método tem a vantagem de uma execução de programa muito rápida, assim que o processo de tradução for concluído. A maioria das implementações de linguagens de produção como C, COBOL e Ada dá-se por meio de compiladores.

A linguagem que um compilador traduz é chamada de linguagem-fonte. O processo de compilação desenvolve-se em diversas etapas, a maioria das quais mostrada na Figura 1.3 (ver p. 42).

O analisador léxico reúne os caracteres do programa-fonte em unidades léxicas que são os identificadores, as palavras especiais, os operadores e os símbolos de pontuação. O analisador léxico ignora os comentários no programa-fonte, porque eles não tem nenhuma utilidade para o compilador.

O analisador sintático pega as unidades do analisador léxico e usa-as para construir estruturas hierárquicas chamadas árvores de análise (*parse trees*), as quais representam a

*N. de T. Teoria dos autômatos: uma disciplina da ciência da computação aberta que diz respeito a um dispositivo abstrato denominado "autômato", o qual executa uma função específica computacional ou de reconhecimento.

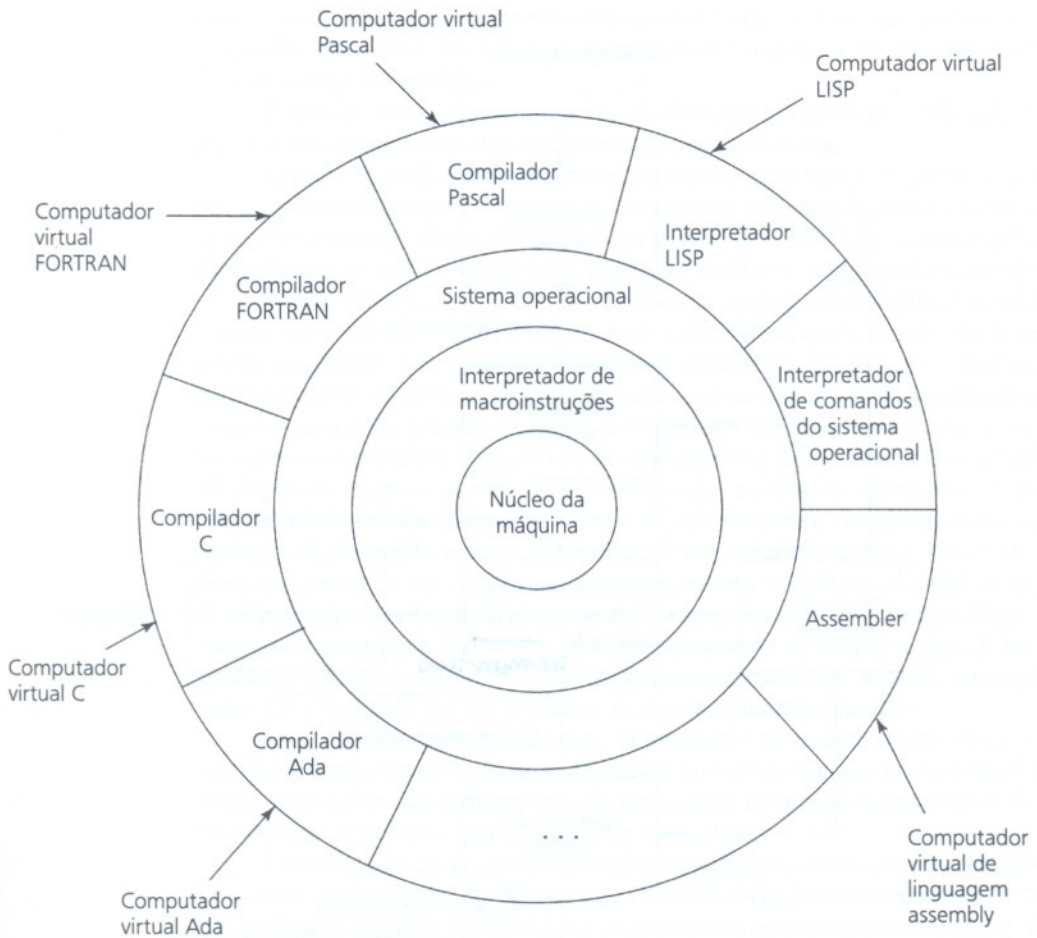


FIGURA 1.2 Interface de computadores virtuais disposta em camadas, fornecida por um computador típico.

estrutura sintática do programa. Em muitos casos, nenhuma estrutura de árvore de análise real é construída; ao contrário, a informação que seria necessária para construí-la é gerada e usada. Tanto as unidades léxicas como as árvores de análise serão discutidas adicionalmente no Capítulo 3.

O gerador de código intermediário produz um programa em uma linguagem diferente, no nível intermediário entre o programa-fonte e a saída final do compilador, o programa em linguagem de máquina (note que as palavras *linguagem* e *código* muitas vezes são usadas de maneira intercambiável). As linguagens intermediárias, às vezes, parecem-se muito com as linguagens *assembly*. Em outros casos, o código intermediário está em um nível bem mais alto do que o *assembly*. O analisador semântico faz parte integralmente do gerador de código intermediário e verifica se há erros difíceis, se não impossíveis, de serem detectados durante a análise sintática, como por exemplo, erros de tipo.

Exemplos simples de análise sintática e de geração de código intermediário são apresentados no Capítulo 3.

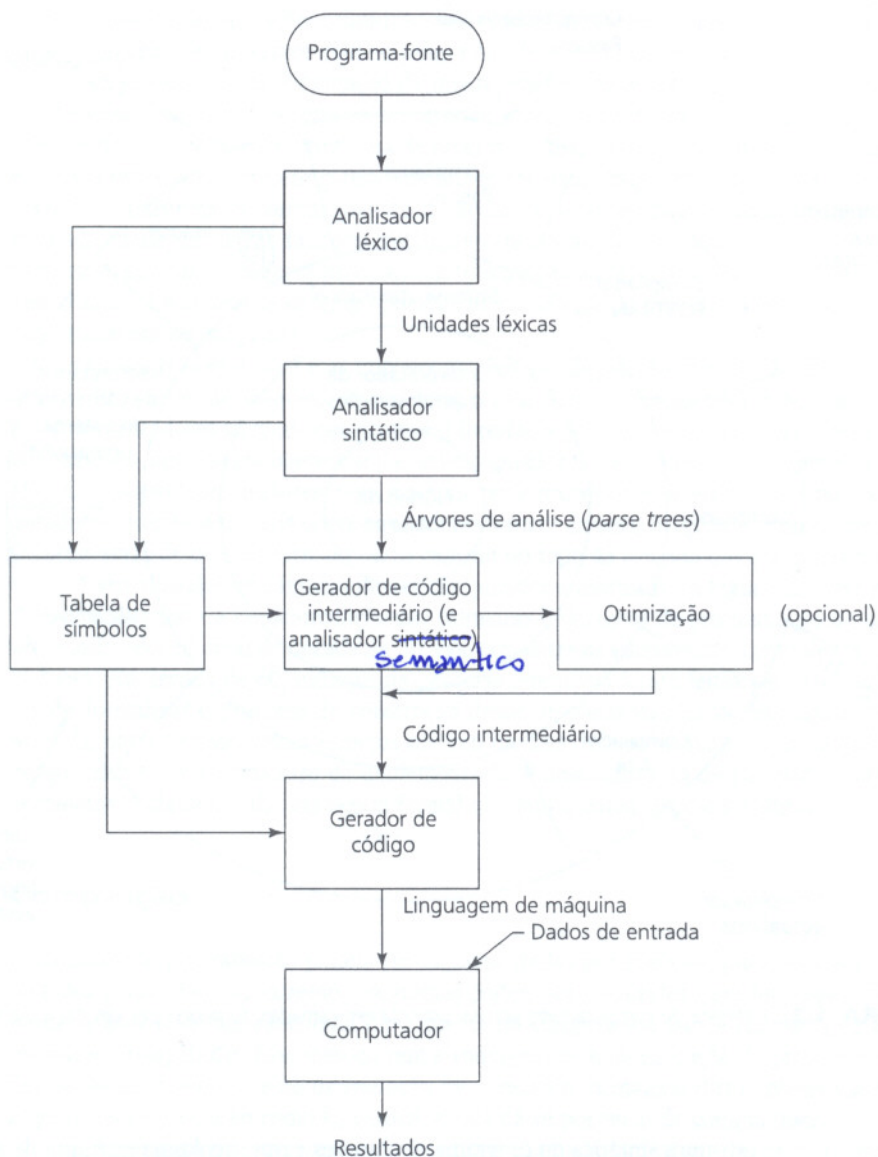


FIGURA 1.3 O processo de compilação.

A otimização, que melhora os programas tornando-os menores ou mais rápidos, ou ambos, muitas vezes, é uma parte opcional da compilação. De fato, alguns compiladores são incapazes de fazer qualquer otimização significativa. Esse tipo de compilador seria usado em situações em que a velocidade de execução do programa traduzido é bem menos importante do que a velocidade de compilação. Um exemplo dessa situação é um laboratório de computação para programadores principiantes. Na maioria das situações comerciais e industriais, a velocidade de execução é mais importante do que a velocidade de compilação.

ção, de modo que a otimização é rotineiramente desejável. Uma vez que muitos tipos de otimização não podem ser feitos em linguagem de máquina, a maioria das otimizações é feita no código intermediário.

O gerador de código converte a versão do código intermediário otimizado do programa para um programa em linguagem de máquina equivalente.

A tabela de símbolos serve como um banco de dados para o processo de compilação. Seu principal conteúdo são informações sobre tipos e atributos de cada nome definido pelo usuário no programa. Essas informações são colocadas na tabela de símbolos pelos analisadores léxico e sintático e usadas pelo analisador semântico e pelo gerador de código.

Conforme afirmou-se acima, não obstante a linguagem de máquina gerada por um compilador possa ser executada diretamente no hardware, quase sempre ela deve ser executada juntamente com algum outro código. A maioria dos programas de usuário também exige programas do sistema operacional. Entre os mais comuns, estão aqueles para entrada (*input*) e para saída (*output*) de dados. O compilador cria chamadas a programas do sistema necessários quando o programa de usuário necessita deles. Antes que os programas em linguagem de máquina produzidos pelo compilador possam ser executados, os programas necessários do sistema operacional devem ser encontrados e vinculados ao do usuário. A operação de vinculação conecta o programa de usuário aos de sistema, colocando os endereços dos pontos de entrada dos programas de sistema nas chamadas a eles no de usuário. O código de usuário e o de sistema juntos, às vezes, são chamados de módulo de carga ou imagem de executável. O processo de coletar programas de sistema e vinculá-los aos programas de usuário é chamado de vinculação e carregamento ou, às vezes, apenas de vinculação. Ele é realizado por um programa de sistema chamado *linkeditor*.

Além dos programas de sistemas, os programas de usuário muitas vezes devem ser vinculados a programas de usuário compilados anteriormente, que residem em bibliotecas. Assim, o *linkeditor* não somente vincula algum dado programa aos programas de sistema, mas também o vincula a outros programas de usuário.

A execução de um programa em código de máquina em um computador com arquitetura von Neumann ocorre em um processo chamado **ciclo buscar-executar** (*fetch-execute cycle*). Como se afirmou na Seção 1.4.1, os programas residem na memória, mas são executados na CPU. Cada instrução a ser executada deve ser transferida da memória para o processador. O endereço da instrução seguinte a ser executada é mantido em um registro chamado contador de programa. O ciclo buscar-executar pode ser descrito de maneira simples pelo seguinte algoritmo:

inicialize o contador de programa

repeat sempre

 busque a instrução apontada pelo contador de programa

 incremente o contador de programa para apontar para a instrução seguinte

 decodifique a instrução

 execute a instrução

end repeat

O passo “decodifique a instrução” no processo acima significa que esta é examinada para determinada ação especificada por ela. A execução do programa encerra-se quando uma instrução de parada é encontrada, ainda que em um computador real uma instrução de parada raramente seja executada. Ao contrário, o controle transfere-se do sistema operacional para um programa de usuário para ser executado e depois retorna ao sistema operacional quando a execução completa-se. Em um computador em que mais de um programa de usuário pode estar na memória em determinado momento, esse processo é bem mais complexo.

A velocidade da conexão entre a memória de um computador e seu processador usualmente determina a velocidade do computador, porque, muitas vezes, as instruções podem ser executadas mais rapidamente do que podem ser transferidas para o processador para serem executadas. Essa conexão é chamada de gargalo de von Neumann; ele é o principal fator limitante na velocidade da arquitetura de computadores de von Neumann. O gargalo de von Neumann foi uma das principais motivações para a pesquisa e para o desenvolvimento de computadores paralelos.

1.7.2 Interpretação Pura

Na extremidade oposta dos métodos de implementação, os programas podem ser interpretados por outro programa chamado interpretador, sem nenhuma conversão. O programa interpretador age como uma simulação de software de uma máquina cujo ciclo buscar-executar lida com instruções de programa em linguagem de alto nível em vez de instruções de máquina. Essa simulação de software, evidentemente, fornece uma máquina virtual para a linguagem.

Essa técnica, chamada de **interpretação pura** ou, simplesmente, de interpretação, tem a vantagem de permitir uma fácil implementação de muitas operações de depuração do código-fonte, porque todas as mensagens de erro em tempo de execução podem referir-se a unidades do código. Por exemplo, se for considerado que um índice de *array* está fora da faixa, a mensagem de erro poderá facilmente indicar a linha da fonte e o nome do *array*. Por outro lado, esse método tem a séria desvantagem de que a execução é de 10 a 100 vezes mais lenta do que em sistemas compilados. A principal causa dessa lentidão é a decodificação das instruções de linguagem de alto nível, bem mais complexas do que as instruções em linguagem de máquina (não obstante possa haver um número menor de comandos do que de instruções em código de máquina equivalente). Portanto, a decodificação de comandos, em vez da conexão entre o processador e a memória, é o gargalo de um interpretador puro.

Outra desvantagem da interpretação pura é que ela freqüentemente exige mais espaço. Além do programa-fonte, a tabela de símbolos deve estar presente na interpretação. Além disso, o programa-fonte deve ser armazenado em uma forma projetada para permitir fácil acesso e modificação, em vez de um tamanho mínimo.

A interpretação é um processo difícil em programas escritos em uma linguagem complicada, porque o significado de cada expressão e instrução deve ser determinado diretamente do programa-fonte em tempo de execução. Linguagens com estruturas mais simples prestam-se à interpretação pura. Por exemplo, a APL e a LIST, às vezes, são implementadas como sistemas interpretativos puros. A maioria dos comandos do sistema operacional, como por exemplo, o conteúdo dos *scripts* do *shell* do UNIX e dos arquivos *.BAT* do DOS, são implementados com interpretadores puros. Linguagens mais complexas, como o FORTRAN e o C, raramente são implementadas com interpretadores puros.

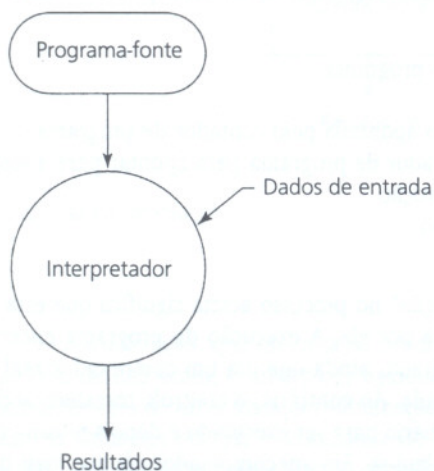


FIGURA 1.4 Interpretação pura.

O processo de interpretação pura é mostrado na Figura 1.4 (ver página anterior).

1.7.3 Sistemas de Implementação Híbridos

Alguns sistemas de implementação de linguagem são um meio-termo entre os compiladores e os interpretadores puros; eles traduzem programas em linguagem de alto nível para uma linguagem intermediária projetada para permitir fácil interpretação. Esse método é mais rápido do que a interpretação pura porque as instruções da linguagem fonte são decodificadas somente uma vez. Essas implementações são chamadas de **sistemas de implementação híbridos**.

O processo usado em um sistema de implementação híbrido é mostrado na Figura 1.5. Em vez de traduzir código em linguagem intermediária para código de máquina, ele simplesmente interpreta o código intermediário.

A Perl é implementada com um sistema híbrido. Ela se desenvolveu a partir da linguagem interpretativa `sh` e `awk`, mas é parcialmente compilada para detectar erros antes da interpretação e para simplificar o interpretador.

As implementações iniciais de Java eram todas híbridas. Sua forma intermediária, chamada código de bytes, oferece portabilidade a qualquer máquina que tenha um interpretador de código de bytes e um sistema de *run-time* associado. Juntos, eles são chamados de Java Virtual Machine. Agora há sistemas que traduzem código de bytes Java para código de máquina para permitir uma execução mais rápida. Porém, os *applets* Java são sempre descarregados do servidor Web na forma de código de bytes.

Às vezes, um implementador pode oferecer tanto implementações compiladas como interpretadas para uma linguagem. Nesses casos, o interpretador é usado para desenvolver e para depurar programas. Então, depois que um estado (relativamente) livre de problemas (*bugs*) é alcançado, os programas são compilados para aumentar sua velocidade de execução.

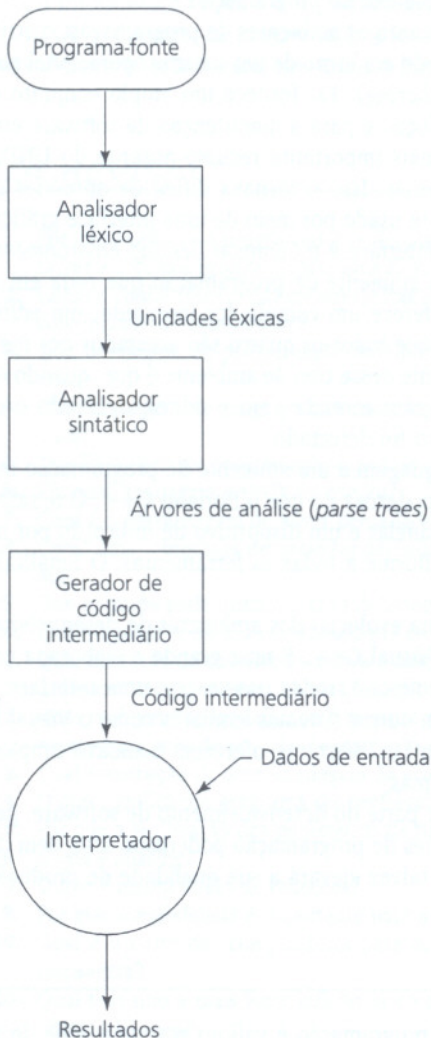


FIGURA 1.5 Sistema de implementação híbrido.

1.8 Ambientes de Programação

Um ambiente de programação é o conjunto de ferramentas usadas no desenvolvimento de software. Esse conjunto pode consistir em somente um sistema de arquivos, em um editor de texto, em um *linker* e em um compilador. Ou pode incluir uma grande coleção de ferramentas integradas, cada uma das quais acessada por meio de uma interface de usuário uniforme. Nesse último caso, o desenvolvimento e a manutenção de software melhorou bastante. Portanto, as características de uma linguagem de programação não são a única medida da capacidade de desenvolvimento de software de um sistema. Descreveremos agora, brevemente, diversos ambientes de programação.

O UNIX é um dos mais antigos ambientes de programação, distribuído pela primeira vez na década de 70, construído em torno de um sistema operacional portátil com compartilhamento de tempo (*time-sharing*). Ele fornece um amplo conjunto de poderosas ferramentas de apoio para a produção e para a manutenção de software em uma variedade de linguagens. No passado, o mais importante recurso ausente do UNIX era uma interface uniforme entre suas ferramentas. Isso o tornava difícil de aprender e de usar. Porém, o UNIX, agora, é freqüentemente usado por meio de uma interface gráfica que roda em cima dele. Em muitos casos, essa interface é o *Common Desktop Environment* (CDE).

O Borland C++ é um ambiente de programação que roda em microcomputadores IBM-PC e compatíveis. Ele oferece um compilador integrado, um editor, um depurador e um sistema de arquivos, em que todos os quatro são acessados por meio de uma interface gráfica. Um recurso conveniente desse tipo de ambiente é que, quando o compilador encontra um erro de sintaxe, ele pára e muda para o editor, deixando o cursor no ponto do programa-fonte em que o erro foi detectado.

O Smalltalk é uma linguagem e um ambiente de programação integrados, mas ela é mais elaborada, complexa e poderosa do que o Borland C++. O Smalltalk foi o primeiro a fazer uso de um sistema de janelas e um dispositivo de indicação por mouse para oferecer ao usuário uma interface uniforme a todas as ferramentas. O Smalltalk será discutido no Capítulo 11.

O passo mais recente na evolução dos ambientes de desenvolvimento de software é representado pelo Microsoft Visual C++. É uma grande e elaborada coleção de ferramentas de desenvolvimento de software, todas usadas por uma interface provida de janelas. Esse sistema, juntamente com outros sistemas similares como o Visual BASIC, o Delphi e o Java Development Kit, da Sun Microsystems, oferecem maneiras simples de construir interfaces gráficas para os programas.

É evidente que a maior parte do desenvolvimento de software, pelo menos no futuro próximo, fará uso de ambientes de programação poderosos. Isso, sem dúvida, aumentará a produtividade do software e talvez elevará a sua qualidade de produção.

RESUMO

O estudo das linguagens de programação é valioso por uma série de importantes razões: aumenta nossa capacidade de usar diferentes construções para escrever programas, possibilita-nos escolher linguagens para projetos de maneira mais inteligente e torna mais fácil a aprendizagem de novas linguagens.

Os computadores são usados em uma ampla variedade de domínios de solução de problemas. O projeto e a avaliação de uma linguagem de programação particular dependem muito do domínio em que eles devem ser usados.

Entre os critérios mais importantes para avaliar linguagens estão a legibilidade, a capacidade de escrita (*writability*), a confiabilidade e o custo global. Esses serão a base na qual examinaremos e julgaremos os vários recursos de linguagem discutidos no restante do livro.

As principais influências sobre o projeto de uma linguagem têm sido a arquitetura de máquina e as metodologias de projeto de software.

Projetar uma linguagem de programação é, antes de mais nada, um feito de engenharia, na qual uma longa lista de *trade-offs* deve ser feita entre os recursos, as construções e as capacidades.

Os principais métodos para implementar linguagens de programação são a compilação, a interpretação pura e a implementação híbrida.

Os ambientes de programação tornaram-se partes importantes dos sistemas de desenvolvimento de software, nos quais a linguagem é apenas um dos componentes.

QUESTÕES DE REVISÃO

1. Por que é útil que o programador tenha algum *background* em projeto de linguagens, ainda que talvez ele jamais projete de fato uma linguagem de programação?
2. Como o conhecimento das características da linguagem de programação pode beneficiar toda a comunidade de computação?
3. Que linguagem de programação dominou a computação científica ao longo dos últimos 35 anos?
4. Que linguagem de programação dominou as aplicações comerciais ao longo dos últimos 35 anos?
5. Que linguagem de programação dominou a inteligência artificial ao longo dos últimos 35 anos?
6. Em qual linguagem o UNIX foi escrito?
7. Qual é a desvantagem de haver demasiados recursos em uma linguagem?
8. Como uma sobrecarga de operador definida pelo usuário prejudica a legibilidade de um programa?
9. Que exemplo pode ilustrar a falta de ortogonalidade no projeto do C?
10. Qual linguagem usou a ortogonalidade como principal critério de projeto?
11. Qual instrução de controle primitiva é usada para construir instruções de controle mais complicadas em linguagens em que elas faltam?
12. Qual problema de legibilidade é causado quando se usa a mesma palavra reservada de fechamento para mais de um tipo de instrução de controle?
13. Qual construção de uma linguagem de programação oferece abstração de processo?
14. O que significa um programa ser confiável?
15. Por que a verificação dos parâmetros de tipo de um subprograma é importante?
16. O que é *aliasing*?
17. O que é manipulação de exceções?
18. Por que a legibilidade é importante para a capacidade de escrita (*writability*)?
19. Qual é o custo dos compiladores para determinada linguagem em relação ao projeto dessa linguagem?
20. Qual tem sido a mais forte influência no projeto de linguagens de programação ao longo dos últimos 40 anos?
21. Qual é o nome da categoria de linguagens de programação cuja estrutura é determinada pela arquitetura de computador de von Neumann?
22. Quais foram as duas deficiências de linguagem de programação descobertas em consequência da pesquisa em desenvolvimento de software na década de 70?
23. Quais são os três recursos fundamentais de uma linguagem de programação orientada a objeto?
24. Qual linguagem foi a primeira a suportar os três recursos fundamentais da programação orientada a objeto?

25. Dê um exemplo de dois critérios de projeto de linguagem que estão em conflito direto um com o outro.
26. Quais são os três métodos gerais para implementar uma linguagem de programação?
27. O que produz uma execução de programa mais rápida: um compilador ou um interpretador puro?
28. Qual papel a tabela de símbolos desempenha em um compilador?
29. O que um *linkeditor* faz?
30. Por que o gargalo de von Neumann é importante?
31. Quais são as vantagens de implementar uma linguagem com um interpretador puro?
32. Qual desvantagem o UNIX tem como ambiente de desenvolvimento de software?

PROBLEMAS

1. Você acredita que nossa capacidade de pensar é influenciada por nossa linguagem? Sustente sua opinião.
2. Quais recursos de linguagens de programação específicas você conhece, cujos fundamentos lógicos são um mistério para você?
3. Quais argumentos você poderia levantar a favor da idéia de uma única linguagem para todos os domínios de programação?
4. Quais argumentos você poderia levantar contra a idéia de uma única linguagem para todos os domínios de programação?
5. Cite e explique outro critério pelo qual as linguagens podem ser julgadas (além daqueles apresentados neste capítulo).
6. Qual instrução de linguagem de programação comum, em sua opinião, é a mais prejudicial para a legibilidade?
7. O Modula-2 usa END para marcar o final de todas as instruções compostas. Quais os argumentos que podem ser levantados contra tal uso?
8. Algumas linguagens, notavelmente o C e o Java, fazem distinção entre maiúsculas e minúsculas nos identificadores. Quais são os prós e os contras nessa decisão de projeto?
9. Explique os diferentes aspectos do custo de uma linguagem de programação.
10. Quais são os argumentos para escrever programas eficientes não obstante o hardware ser relativamente barato?
11. Descreva alguns *trade-offs* de projeto entre eficiência e segurança em algumas linguagens que você conhece.
12. Quais os principais recursos que uma linguagem de programação perfeita incluiria, em sua opinião?
13. A primeira linguagem de programação de alto nível que você aprendeu foi implementada com um interpretador puro, com um sistema de implementação híbrido ou com um compilador? (Você não saberia isso necessariamente sem pesquisa.)
14. Descreva as vantagens e as desvantagens de algum ambiente de programação que você usou.
15. Como as instruções de declaração de tipo para variáveis simples afetam a legibilidade de uma linguagem, considerando que algumas linguagens não as exigem?
16. Escreva uma avaliação de alguma linguagem de programação que conheça, usando os critérios descritos neste capítulo.
17. O Pascal usa o ponto e vírgula para separar instruções, enquanto o C usa-o para finalizar instruções. Qual dessas, em sua opinião, é a mais natural e a que tem menos probabilidade de resultar em erros de sintaxe? Sustente sua resposta.
18. Algumas linguagens, como o Pascal e o C, usam delimitadores em ambas as extremidades dos comentários. Outras linguagens, como o FORTRAN e a Ada, usam um símbolo ou um par de símbolos para indicar o início de um comentário e o final da linha para finalizá-lo. Discuta as vantagens e as desvantagens de cada opção de projeto com respeito aos nossos critérios.